
Sebenta 03

Virtualização de Sistemas

Infraestruturas de Computação na Nuvem | Aula 03

Objeto e utilização deste documento

O presente documento acompanha a apresentação da Aula 03. Cada página reproduz um slide e desenvolve, em texto corrido, a matéria nele condensada. A aula integra sete questões de escolha múltipla, cuja resolução é apresentada no slide imediatamente seguinte, e conclui com o enquadramento do laboratório de containers, percorrido em equipamento próprio durante a aula.

Virtualização de Sistemas

Infraestruturas de Computação na Nuvem, Aula 03

Catarina Silva

ESTGA, Universidade de Aveiro | 2026

A Aula 03 introduz o mecanismo que torna operacional a partilha de recursos estabelecida conceptualmente na Aula 01 e a infraestrutura física analisada na Aula 02. A exposição percorre a fundamentação teórica da virtualização, as três estratégias históricas de a realizar em arquitetura x86, e a sua concretização nos domínios da memória, do I/O e do armazenamento. Conclui com a distinção entre virtualização completa e virtualização ao nível do sistema operativo, que o laboratório associado torna observável em equipamento próprio.

Agenda (1/3)

- Conceito de virtualização e níveis de aplicação.
- Requisitos de Popek e Goldberg e classificação de instruções.
- Anéis de privilégio e a limitação da arquitetura x86.
- Estratégias de virtualização de CPU e arquiteturas de hypervisor.

A primeira parte estabelece o enquadramento teórico. Define virtualização, distingue os níveis a que se aplica, e apresenta o resultado formal que determina quando uma arquitetura de processador admite a construção de um monitor de máquina virtual. Conclui com as estratégias adotadas quando essa condição não se verifica.

Agenda (2/3)

- Virtualização de memória: tabelas sombra e paginação em dois níveis.
- Sobrecompromisso de memória: KSM e ballooning.
- Virtualização de I/O: emulação, virtio, passthrough e SR-IOV.
- Armazenamento virtual: formatos, provisionamento fino e instantâneos.

A segunda parte trata da virtualização dos recursos que não o processador. A memória, cujo tratamento constituiu historicamente a maior fonte de sobrecarga; o I/O, onde persiste a maior distância face ao desempenho nativo; e o armazenamento, com os seus formatos e mecanismos de instantâneo.

Agenda (3/3)

- Migração em execução e sobrecarga de virtualização.
- KVM, LXC e a plataforma Proxmox VE.
- Virtualização completa e virtualização ao nível do sistema operativo.
- Enquadramento do laboratório de containers.

A terceira parte aborda a operação corrente de infraestrutura virtualizada, designadamente a migração de instâncias entre anfitriões e a sobrecarga a esperar. Apresenta as tecnologias concretas utilizadas na unidade curricular e enquadra o laboratório.

Conceito de Virtualização

- Designa-se por **virtualização** a construção de uma representação abstrata de um recurso computacional, funcionalmente equivalente ao recurso físico mas dissociada da sua implementação.
- A camada que realiza esta abstração assume três funções: *traduzir* referências virtuais em físicas, *arbitrar* o acesso concorrente e *isolar* as instâncias entre si.
- Propriedades decorrentes: consolidação de cargas sobre menos equipamento, isolamento de falhas entre instâncias e portabilidade face ao hardware subjacente.
- A virtualização constitui o mecanismo que torna operacional o modelo de partilha de recursos estabelecido na Aula 01.

A definição adotada é deliberadamente geral: virtualização designa a construção de uma representação abstrata de um recurso computacional, funcionalmente equivalente ao recurso físico mas dissociada da sua implementação. A generalidade é intencional, dado que o conceito não se restringe a máquinas virtuais. A memória virtual de um sistema operativo, um sistema de ficheiros e o endereçamento de rede constituem instâncias do mesmo princípio. A camada que realiza a abstração assume sempre três funções: traduzir referências virtuais em físicas, arbitrar o acesso concorrente e isolar as instâncias entre si. A falha de qualquer uma compromete a abstração e produz comportamento inexplicável a partir da interface apresentada.

Níveis de Virtualização

- **Nível de hardware:** o hypervisor apresenta um conjunto de instruções e dispositivos virtuais. Cada instância executa núcleo próprio e sistema operativo completo. Exemplos: KVM, Xen, ESXi.
- **Nível de sistema operativo:** o núcleo é único e partilhado, apresentando a cada instância uma vista isolada dos seus recursos. Exemplos: LXC, Docker, Solaris Zones.
- **Nível de aplicação:** a abstração situa-se sobre o sistema operativo, executando código destinado a uma máquina abstrata. Exemplos: JVM, CLR.
- O critério distintivo é a *fronteira de duplicação*: o que é replicado por instância e o que permanece partilhado. Determina a sobrecarga e o grau de isolamento.

A virtualização aplica-se a três níveis, cuja distinção assenta na fronteira de duplicação, isto é, no que é replicado por instância e no que permanece partilhado. Ao nível do hardware, o hypervisor apresenta instruções e dispositivos virtuais, executando cada instância núcleo próprio e sistema operativo completo. Ao nível do sistema operativo, o núcleo é único e partilhado, sendo apresentada a cada instância uma vista isolada dos recursos. Ao nível da aplicação, a abstração situa-se sobre o sistema operativo e executa código destinado a uma máquina abstrata, como sucede na JVM. A fronteira de duplicação determina simultaneamente a sobrecarga incorrida e o grau de isolamento obtido, sendo as duas grandezas inversamente relacionadas.

Os Requisitos de Popek e Goldberg

- Formulados em 1974, estabelecem três propriedades exigíveis a um monitor de máquina virtual.
- **Equivalência:** um programa executado sob o monitor apresenta comportamento indistinguível do que apresentaria em execução direta sobre o hardware.
- **Controlo de recursos:** o monitor mantém domínio completo sobre os recursos físicos, não podendo uma instância aceder a recursos não atribuídos.
- **Eficiência:** uma fração estatisticamente dominante das instruções executa diretamente no processador, sem intervenção do monitor.
- O terceiro requisito distingue virtualização de emulação: um emulador satisfaz os dois primeiros, mas interpreta a totalidade das instruções.

Os requisitos formulados por Popek e Goldberg em 1974 permanecem o critério de referência para avaliar qualquer solução de virtualização. A equivalência exige que um programa executado sob o monitor apresente comportamento indistinguível do que apresentaria em execução direta. O controlo de recursos exige que o monitor mantenha domínio completo sobre o hardware, não podendo uma instância aceder a recursos não atribuídos. A eficiência exige que uma fração estatisticamente dominante das instruções execute diretamente no processador. Este terceiro requisito é o que distingue virtualização de emulação: um emulador satisfaz os dois primeiros, mas interpreta a totalidade das instruções, incorrendo em penalização de desempenho de ordem de grandeza.

Classificação de Instruções

- **Privilegiadas:** geram exceção quando executadas em modo utilizador, sendo admitidas apenas em modo supervisor.
- **Sensíveis:** subdividem-se em sensíveis ao *controlo*, que alteram a configuração de recursos do sistema, e sensíveis ao *comportamento*, cujo resultado depende dessa configuração.
- **Inócuas:** as restantes, sem efeito sobre o estado global, executáveis sem mediação.
- **Teorema de Popek e Goldberg:** um monitor de máquina virtual pode ser construído se o conjunto das instruções sensíveis estiver contido no das privilegiadas. A condição é verificável sobre a especificação da arquitetura, independentemente de qualquer implementação.

A classificação das instruções fundamenta o resultado central desta parte da aula. Dizem-se privilegiadas as instruções que geram exceção quando executadas em modo utilizador. Dizem-se sensíveis as que alteram a configuração de recursos do sistema, caso das sensíveis ao controlo, ou cujo resultado depende dessa configuração, caso das sensíveis ao comportamento. As restantes, sem efeito sobre o estado global, dizem-se inócuas. O teorema estabelece que um monitor pode ser construído se o conjunto das instruções sensíveis estiver contido no das privilegiadas, condição que garante a interceção de toda a operação com efeito global. Trata-se de uma propriedade da especificação da arquitetura, verificável independentemente de qualquer implementação concreta.

Anéis de Privilégio e a Limitação do x86

- A arquitetura x86 define quatro anéis de privilégio, do anel 0, reservado ao núcleo, ao anel 3, destinado a aplicações.
- Sob virtualização, o anel 0 é ocupado pelo monitor, sendo o núcleo convidado deslocado para um anel de menor privilégio, técnica designada por *ring deprivileging*.
- A arquitetura x86 anterior a 2005 não satisfazia a condição do teorema: dezassete instruções eram sensíveis sem serem privilegiadas.
- Exemplo característico: POPF, que ao restaurar o registo de estado em anel 3 ignora silenciosamente a alteração do bit de interrupções. A falha é adversa por não ser detetável: a instrução não falha, produz resultado incorreto.

A arquitetura x86 define quatro anéis de privilégio, sendo o anel zero reservado ao núcleo e o anel três destinado a aplicações. Sob virtualização, o anel zero passa a ser ocupado pelo monitor e o núcleo convidado é deslocado para um anel de menor privilégio, técnica designada por *ring deprivileging*. A dificuldade histórica reside noutro ponto: a arquitetura x86 anterior a 2005 não satisfazia a condição do teorema, existindo dezassete instruções sensíveis que não eram privilegiadas. O exemplo característico é a instrução POPF que, ao restaurar o registo de estado em anel três, ignora silenciosamente a alteração do bit de interrupções em vez de gerar exceção. A natureza da falha é particularmente adversa: a instrução não falha, produz resultado incorreto sem qualquer sinalização.

Trap-and-Emulate

- Mecanismo base da virtualização em arquiteturas que satisfazem a condição do teorema.
- O código convidado executa diretamente no processador. Ao emitir uma instrução privilegiada em modo não privilegiado, o processador gera exceção e transfere o controlo para o monitor.
- O monitor determina a operação pretendida, emula o seu efeito sobre o estado virtual da instância e devolve o controlo ao ponto de interrupção.
- Aplica-se a múltiplos subsistemas: registos de controlo do processador, unidade de gestão de memória, controlador de interrupções, temporizadores e dispositivos periféricos.

O trap-and-emulate constitui o mecanismo base da virtualização em arquiteturas que satisfazem a condição do teorema. O código convidado executa diretamente no processador e, ao emitir uma instrução privilegiada em modo não privilegiado, provoca uma exceção que transfere o controlo para o monitor. Este determina a operação pretendida, emula o seu efeito sobre o estado virtual da instância e devolve o controlo ao ponto de interrupção, sem que o convidado detete a intervenção. O mecanismo aplica-se de modo uniforme a registos de controlo do processador, unidade de gestão de memória, controlador de interrupções, temporizadores e dispositivos periféricos.

Pergunta 1

Aplicabilidade do teorema

Numa arquitetura hipotética existem instruções que consultam a configuração de recursos do sistema e devolvem resultado distinto consoante o modo de execução, sem gerar exceção em modo utilizador. Que consequência decorre para a construção de um monitor por trap-and-emulate?

- A) Nenhuma. O monitor interceta igualmente essas instruções.
- B) O mecanismo é insuficiente: tais instruções são sensíveis sem serem privilegiadas, pelo que não geram exceção e produzem resultado incorreto sem sinalização.
- C) O monitor deve ser executado em anel 3 para as intercetar.
- D) A arquitetura não admite qualquer forma de virtualização.

A primeira questão examina a consequência de uma arquitetura não satisfazer a condição do teorema. Recomenda-se a formulação de resposta antes da consulta do slide seguinte.

Resposta 1

Opção correta: B

A situação descrita corresponde à violação da condição do teorema: existem instruções sensíveis fora do conjunto das privilegiadas. Não gerando exceção, não são interceptáveis por trap-and-emulate, e a instância obtém informação relativa ao estado real da máquina em vez do seu estado virtual, comprometendo o requisito de equivalência. A opção D é incorreta: a virtualização permanece possível por outras vias, designadamente tradução binária ou paravirtualização, apresentadas em seguida.

A fundamentação consta do slide. Importa assinalar que a conclusão não é a impossibilidade de virtualizar a arquitetura, mas apenas a insuficiência do mecanismo de trap-and-emulate isolado. Os três slides seguintes apresentam precisamente as estratégias desenvolvidas para contornar esta limitação, todas historicamente motivadas pelo caso concreto do x86.

Estratégias de Virtualização de CPU

- **Virtualização completa por tradução binária:** o monitor analisa e reescreve o fluxo de instruções do convidado antes da execução, sem exigir alteração do sistema operativo.
- **Paravirtualização:** o núcleo convidado é modificado para invocar explicitamente o monitor nas operações sensíveis, dispensando a intercepção.
- **Virtualização assistida por hardware:** extensões do processador introduzem um modo de execução dedicado, restaurando a validade do trap-and-emulate.
- As três respondem à mesma limitação arquitetural, com compromissos distintos entre desempenho, compatibilidade e necessidade de alterar o convidado.

Três estratégias respondem à mesma limitação arquitetural, com compromissos distintos. A virtualização completa por tradução binária analisa e reescreve o fluxo de instruções do convidado antes da execução, sem exigir alteração do sistema operativo. A paravirtualização modifica o núcleo convidado para que invoque explicitamente o monitor nas operações sensíveis, dispensando a intercepção. A virtualização assistida por hardware introduz um modo de execução dedicado no processador, restaurando a validade do trap-and-emulate. As três diferem essencialmente em dois eixos: a necessidade de modificar o convidado e a necessidade de suporte específico do processador.

Tradução Binária Dinâmica

- O monitor examina blocos de código do convidado antes da execução e substitui as instruções problemáticas por sequências equivalentes que transferem o controle.
- A tradução incide apenas sobre código executado em anel 0. O código de aplicação, em anel 3, executa sem modificação.
- Os blocos traduzidos são retidos em memória cache, pelo que o custo é amortizado: código executado repetidamente é traduzido uma única vez.
- Vantagem determinante: admite sistemas operativos convidados não modificados, incluindo sistemas de código fechado. Foi a abordagem que viabilizou comercialmente a virtualização de x86.

A tradução binária dinâmica opera sobre blocos de código do convidado antes da sua execução, substituindo as instruções problemáticas por sequências equivalentes que transferem o controle ao monitor. Dois aspectos explicam a sua viabilidade prática. Primeiro, a tradução incide apenas sobre código executado em anel zero, permanecendo o código de aplicação, largamente dominante em volume, executado sem modificação. Segundo, os blocos traduzidos são retidos em cache, pelo que código executado repetidamente é traduzido uma única vez e o custo amortiza-se. A vantagem determinante consiste em admitir sistemas operativos convidados não modificados, incluindo sistemas de código fechado, o que explica ter sido esta a abordagem que viabilizou comercialmente a virtualização de x86.

Paravirtualização

- O núcleo convidado é modificado de modo a não emitir instruções sensíveis, substituindo-as por invocações explícitas ao monitor, designadas **hypercalls**.
- A interface entre convidado e monitor passa a ser declarada, à semelhança de uma chamada de sistema entre aplicação e núcleo.
- Elimina-se o custo de interceção e de emulação, e o monitor deixa de necessitar de emular hardware que não existe.
- Limitação: exige acesso ao código do sistema operativo convidado e adaptação a cada monitor, o que restringe a sua aplicação a sistemas de código aberto.
- Persiste hoje sobretudo nos controladores paravirtualizados de I/O, matéria retomada adiante.

A paravirtualização parte de premissa distinta: admite-se que o núcleo convidado pode ser modificado. Elimina-se assim a emissão de instruções sensíveis, substituídas por invocações explícitas ao monitor designadas hypercalls. A interface entre convidado e monitor torna-se declarada, à semelhança da interface entre aplicação e núcleo. As vantagens são apreciáveis: elimina-se o custo de interceção e de emulação, e o monitor deixa de necessitar de emular hardware inexistente. A limitação é igualmente clara: exige acesso ao código do sistema operativo convidado e adaptação a cada monitor, o que restringe a aplicação a sistemas de código aberto. A abordagem persiste hoje sobretudo sob forma parcial, nos controladores paravirtualizados de I/O tratados adiante.

Virtualização Assistida por Hardware

- As extensões Intel VT-x e AMD-V introduzem dois modos de operação ortogonais aos anéis: **VMX root**, onde executa o monitor, e **VMX non-root**, onde executa o convidado.
- O núcleo convidado executa em anel 0 do modo non-root, pelo que não é necessário deslocá-lo de anel nem reescrever o seu código.
- A transição entre modos é mediada por uma estrutura de controlo em memória, a **VMCS**, que retém o estado de ambos os contextos e determina que eventos provocam saída para o monitor.
- Designa-se por **VM exit** a transição para o monitor e por **VM entry** o regresso. Cada transição tem custo mensurável, pelo que a otimização consiste em reduzir a sua frequência.

As extensões Intel VT-x e AMD-V resolvem a questão na origem, introduzindo dois modos de operação ortogonais aos anéis de privilégio. O monitor executa em modo VMX root e o convidado em modo VMX non-root, dispondo este último igualmente de quatro anéis. Em consequência, o núcleo convidado executa em anel zero do seu próprio modo, dispensando tanto o deslocamento de anel como a reescrita de código. A transição entre modos é mediada por uma estrutura em memória, a VMCS, que retém o estado de ambos os contextos e determina que eventos provocam saída para o monitor. Designa-se VM exit a transição para o monitor e VM entry o regresso. Cada transição tem custo mensurável, pelo que a otimização de desempenho em virtualização moderna consiste, em larga medida, em reduzir a sua frequência.

Pergunta 2

Seleção de estratégia

Pretende-se virtualizar um sistema operativo de código fechado, sem possibilidade de modificação, sobre processadores anteriores à introdução das extensões de virtualização. Que estratégia é aplicável?

- A) Paravirtualização, por apresentar melhor desempenho.
- B) Trap-and-emulate puro, por ser o mecanismo de referência.
- C) Tradução binária dinâmica, única das três que dispensa simultaneamente a modificação do convidado e o suporte do processador.
- D) Nenhuma. O cenário descrito não admite virtualização.

A segunda questão solicita a seleção da estratégia aplicável a um cenário com duas restrições simultâneas. A resolução requer o confronto das três estratégias com ambas as restrições enunciadas.

Resposta 2

Opção correta: C

A paravirtualização exige modificação do núcleo convidado, inviável em código fechado. O trap-and-emulate puro pressupõe a validade da condição do teorema, que a arquitetura em causa não satisfaz. As extensões de hardware não se encontram disponíveis. Resta a tradução binária dinâmica, que opera sobre o fluxo de instruções em tempo de execução sem requerer cooperação do convidado nem suporte do processador. Foi precisamente esta a razão da sua adoção comercial na década de 2000.

A fundamentação consta do slide anterior. O exercício ilustra o valor histórico da tradução binária: foi a única das três estratégias aplicável ao parque instalado antes de 2005, sem cooperação dos fabricantes de sistemas operativos. A generalização das extensões de hardware reduziu-lhe a importância, mas o raciocínio permanece pertinente sempre que se avalia uma solução de virtualização face às restrições concretas do ambiente.

Arquiteturas de Hypervisor

- **Tipo 1**, nativo: executa diretamente sobre o hardware, assumindo as funções de escalonamento e gestão de recursos. Exemplos: VMware ESXi, Xen, KVM.
- **Tipo 2**, alojado: executa como processo sobre um sistema operativo anfitrião, que mantém a mediação do acesso ao hardware. Exemplos: VirtualBox, VMware Workstation.
- A distinção reduz-se ao número de camadas interpostas entre a instância e o hardware, com efeito direto sobre a sobrecarga e sobre o determinismo do escalonamento.
- O KVM constitui caso particular: sendo módulo do núcleo Linux, converte o próprio núcleo em hypervisor de tipo 1, mantendo embora as funcionalidades de sistema operativo convencional.

A distinção entre hypervisors de tipo um e de tipo dois reduz-se ao número de camadas interpostas entre a instância e o hardware. O tipo um executa diretamente sobre o hardware e assume as funções de escalonamento e gestão de recursos. O tipo dois executa como processo sobre um sistema operativo anfitrião, que mantém a mediação do acesso ao hardware. A consequência é dupla: maior sobrecarga e menor determinismo do escalonamento no tipo dois, dado que o hypervisor concorre com os restantes processos do anfitrião. O KVM constitui caso particular que merece nota, por ser módulo do núcleo Linux: converte o próprio núcleo em hypervisor de tipo um, mantendo embora todas as funcionalidades de sistema operativo convencional.

As Duas Arquiteturas de Hypervisor



O diagrama representa as duas arquiteturas em confronto. No tipo um, o percurso entre a instância e o hardware atravessa uma única camada. No tipo dois, atravessa duas, o hypervisor e o sistema operativo anfitrião. Esta diferença de profundidade explica a predominância do tipo um em ambientes de produção, onde cada camada representa desempenho perdido, e a persistência do tipo dois em postos de trabalho, onde a conveniência de executar sobre um sistema já instalado prevalece sobre o desempenho máximo.

Virtualização de Memória: o Problema

- Um sistema operativo convidado assume dispor de memória física contígua com início no endereço zero, e mantém tabelas de páginas que traduzem endereços virtuais nesse espaço.
- Sob virtualização, esse espaço é ele próprio virtual. Passam a existir três espaços de endereçamento: virtual do convidado, físico do convidado e físico do anfitrião.
- A unidade de gestão de memória do processador realiza uma única tradução, pelo que a composição das duas restantes tem de ser assegurada pelo monitor ou pelo hardware.
- Constitui a fonte de sobrecarga mais significativa da virtualização anterior ao suporte dedicado.

A virtualização de memória coloca uma dificuldade específica que merece exposição cuidada. Um sistema operativo convidado assume dispor de memória física contígua com início no endereço zero, e mantém tabelas de páginas que traduzem endereços virtuais nesse espaço. Sob virtualização, porém, esse espaço é ele próprio virtual, pelo que passam a existir três espaços de endereçamento em vez de dois: o virtual do convidado, o físico do convidado e o físico do anfitrião. A dificuldade decorre de a unidade de gestão de memória do processador realizar uma única tradução, tendo a composição das restantes de ser assegurada por outra via. Constituiu historicamente a fonte de sobrecarga mais significativa da virtualização.

Tabelas Sombra e Paginação em Dois Níveis

- **Tabelas sombra:** o monitor mantém tabelas próprias que traduzem diretamente o endereço virtual do convidado no endereço físico do anfitrião, compondo antecipadamente as duas traduções.
- O custo reside na manutenção: qualquer alteração às tabelas do convidado tem de ser intercetada e propagada, o que exige marcar essas páginas como apenas de leitura e tratar as exceções resultantes.
- **EPT** (Intel) e **NPT** (AMD) transferem a composição para o hardware, que percorre duas hierarquias de tabelas sucessivamente, sem intervenção do monitor.
- A travessia passa a ser bidimensional e, em caso de falha de TLB, mais dispendiosa. Em contrapartida, elimina-se a interceção de escritas, largamente dominante na prática.

Duas soluções sucederam-se. As tabelas sombra consistem em tabelas mantidas pelo monitor que traduzem diretamente o endereço virtual do convidado no endereço físico do anfitrião, compondo antecipadamente as duas traduções. O custo reside na manutenção: qualquer alteração às tabelas do convidado tem de ser intercetada e propagada, o que obriga a marcar essas páginas como apenas de leitura e a tratar as exceções resultantes, operação frequente e dispendiosa. As extensões EPT na Intel e NPT na AMD transferem a composição para o hardware, que percorre duas hierarquias de tabelas sucessivamente. A travessia torna-se bidimensional e, em caso de falha de TLB, mais dispendiosa, mas elimina-se a interceção de escritas, largamente dominante na prática. O balanço é francamente favorável à segunda solução.

Sobrecompromisso de Memória: KSM e Ballooning

- A soma da memória atribuída às instâncias excede frequentemente a memória física instalada, prática admissível por raramente ser utilizada na totalidade em simultâneo.
- **KSM** (*Kernel Samepage Merging*): o anfitrião identifica periodicamente páginas de conteúdo idêntico entre instâncias e substitui-as por uma única página física partilhada, com cópia na escrita.
- **Ballooning**: um controlador instalado no convidado solicita memória ao seu próprio sistema operativo, que a liberta de cache, devolvendo-a ao anfitrião sem que o convidado a considere perdida.
- Ambos exigem cooperação e apresentam custo: o KSM consome processador na comparação de páginas; o ballooning depende da presença do controlador no convidado.

A soma da memória atribuída às instâncias excede frequentemente a memória física instalada, prática admissível por raramente ser utilizada na totalidade em simultâneo. Dois mecanismos a suportam. O KSM identifica periodicamente páginas de conteúdo idêntico entre instâncias e substitui-as por uma única página física partilhada, com cópia na escrita caso alguma instância a modifique. O ballooning recorre a um controlador instalado no convidado que solicita memória ao próprio sistema operativo deste, o qual a liberta de cache, devolvendo-a ao anfitrião sem que o convidado a considere perdida. Ambos exigem cooperação e apresentam custo: o KSM consome processador na comparação de páginas, e o ballooning depende da presença do controlador no convidado.

Pergunta 3

Efeito do sobrecompromisso

Um anfitrião dispõe de 64 GB de memória física e aloja vinte instâncias com 4 GB atribuídos cada, totalizando 80 GB. As instâncias executam a mesma imagem de sistema operativo. Que mecanismo explica a viabilidade desta configuração?

- A) O hypervisor comprime a memória de todas as instâncias.
- B) As instâncias utilizam habitualmente menos do que o atribuído, e o KSM consolida as páginas idênticas resultantes da imagem comum.
- C) A configuração é inviável e o anfitrião recusará iniciar a vigésima instância.
- D) O excesso é integralmente transferido para disco, sem consequências.

A terceira questão examina a viabilidade de uma configuração com sobrecompromisso de memória, solicitando a identificação do mecanismo que a sustenta. Os dados do enunciado, designadamente a execução de imagem comum, são determinantes para a resposta.

Resposta 3

Opção correta: B

A atribuição constitui um limite superior e não uma reserva efetiva, pelo que o consumo real é tipicamente inferior. Executando as instâncias a mesma imagem, uma fração substancial das suas páginas, designadamente código de núcleo e bibliotecas partilhadas, apresenta conteúdo idêntico, e o KSM consolida-a numa única página física. A opção D descreve a alternativa de recurso, a transferência para disco, que o anfitrião aplica em situação de esgotamento com degradação acentuada do desempenho, e não como regime normal de operação.

A fundamentação consta do slide anterior. Importa reter a distinção entre atribuição e reserva: a memória atribuída constitui limite superior e não reserva efetiva. Importa igualmente reter que a transferência para disco, descrita na opção D, constitui mecanismo de recurso aplicado em situação de esgotamento, com degradação acentuada do desempenho, e não regime normal de operação.

Virtualização de I/O: Emulação e virtio

- O I/O constitui o domínio de maior sobrecarga: ao contrário do cálculo, envolve acesso a dispositivos e, por conseguinte, transições frequentes para o monitor.
- **Emulação**: o monitor reproduz o comportamento de um dispositivo real conhecido, por exemplo uma placa de rede Intel e1000. Cada acesso a registo do dispositivo provoca uma transição.
- Vantagem: o convidado utiliza controladores próprios, sem qualquer adaptação. Desvantagem: o número de transições torna o desempenho substancialmente inferior ao nativo.
- **virtio**: interface de dispositivo paravirtualizada, na qual o convidado sabe estar virtualizado e comunica por filas partilhadas em memória, agrupando operações e reduzindo as transições em uma a duas ordens de grandeza.

O I/O constitui o domínio de maior sobrecarga da virtualização, por envolver acesso a dispositivos e, em consequência, transições frequentes para o monitor. A emulação reproduz o comportamento de um dispositivo real conhecido, o que permite ao convidado utilizar controladores próprios sem qualquer adaptação, mas implica uma transição por cada acesso a registo do dispositivo. O virtio adota abordagem distinta: constitui uma interface de dispositivo paravirtualizada na qual o convidado sabe estar virtualizado e comunica por filas partilhadas em memória. O agrupamento de operações que daqui resulta reduz as transições em uma a duas ordens de grandeza, o que explica ser esta a configuração recomendada em qualquer instância que não exija compatibilidade com controladores legados.

Virtualização de I/O: Passthrough e SR-IOV

- **Passthrough:** um dispositivo físico é atribuído em exclusivo a uma instância, que o opera com o controlador nativo e desempenho equivalente ao nativo.
- Requer unidade de gestão de memória de I/O (IOMMU) para confinar os acessos diretos à memória da instância. Implica a perda do dispositivo pelo anfitrião e impede a migração em execução.
- **SR-IOV:** extensão do PCI Express pela qual um dispositivo se apresenta como uma função física, gerida pelo anfitrião, e múltiplas funções virtuais, atribuíveis individualmente a instâncias distintas.
- Combina o desempenho do passthrough com a partilha do dispositivo, ao custo de suporte específico no equipamento. Aplicação típica em placas de rede de alto débito e aceleradores.

Duas abordagens adicionais dispensam a mediação do monitor. O passthrough atribui um dispositivo físico em exclusivo a uma instância, que o opera com o controlador nativo e desempenho equivalente ao nativo. Exige unidade de gestão de memória de I/O para confinar os acessos diretos à memória da instância, e apresenta duas restrições relevantes: o anfitrião perde o dispositivo e a instância deixa de ser migrável. O SR-IOV supera a primeira restrição ao apresentar o dispositivo como uma função física, gerida pelo anfitrião, e múltiplas funções virtuais atribuíveis individualmente a instâncias distintas. Combina assim o desempenho do passthrough com a partilha do dispositivo, ao custo de suporte específico no equipamento, disponível sobretudo em placas de rede de alto débito e aceleradores.

Virtualização de Rede

- Cada instância dispõe de uma ou mais interfaces virtuais, ligadas a um **comutador virtual** implementado em software no anfitrião.
- O comutador virtual reproduz as funções de um comutador físico, designadamente aprendizagem de endereços, segmentação por VLAN e aplicação de políticas de tráfego.
- O tráfego entre instâncias do mesmo anfitrião não alcança a rede física, o que reduz a latência mas subtrai esse tráfego à observação dos instrumentos de monitorização de rede convencionais.
- A programação centralizada destes comutadores constitui o objeto da Aula 06.

Cada instância dispõe de uma ou mais interfaces virtuais ligadas a um comutador virtual implementado em software no anfitrião. Este reproduz as funções de um comutador físico, designadamente aprendizagem de endereços, segmentação por VLAN e aplicação de políticas de tráfego. Uma consequência operacional merece registo: o tráfego entre instâncias do mesmo anfitrião não alcança a rede física, o que reduz a latência mas subtrai esse tráfego à observação dos instrumentos de monitorização de rede convencionais, com implicações para deteção de intrusão e auditoria. A programação centralizada destes comutadores constitui o objeto da Aula 06.

Formatos de Disco Virtual

- **raw**: representação direta do conteúdo do disco, sem metadados. Desempenho máximo e ausência de funcionalidades acessórias.
- **qcow2**: formato nativo do QEMU, com provisionamento fino, instantâneos internos, compressão e cifra. A indireção introduzida tem custo de desempenho mensurável.
- **VMDK e VHDX**: formatos equivalentes nos ecossistemas VMware e Microsoft.
- **Aprovisionamento fino**: o ficheiro ocupa em armazenamento apenas os blocos efetivamente escritos, e não a capacidade declarada ao convidado.

O formato raw constitui representação direta do conteúdo do disco, sem metadados, oferecendo desempenho máximo e ausência de funcionalidades acessórias. O qcow2, formato nativo do QEMU, acrescenta provisionamento fino, instantâneos internos, compressão e cifra, ao custo de uma indireção com efeito mensurável no desempenho. Os formatos VMDK e VHDX desempenham papel equivalente nos ecossistemas VMware e Microsoft. O provisionamento fino merece destaque autónomo: o ficheiro ocupa em armazenamento apenas os blocos efetivamente escritos, e não a capacidade declarada ao convidado, o que admite a prática analisada no slide seguinte.

Aprovisionamento Fino: Risco Operacional

- O provisionamento fino admite declarar capacidade agregada superior à fisicamente instalada, prática análoga ao sobrecompromisso de memória.
- A diferença é determinante: a memória é reclamável, o armazenamento escrito não. Uma instância que preencha o seu disco não liberta espaço ao terminar.
- Consequência do esgotamento do volume subjacente: as escritas falham em todas as instâncias que dele dependem, e não apenas na responsável pelo consumo.
- Requisito operacional: monitorização do espaço real disponível com limiar de alerta, e não do espaço declarado às instâncias.

O provisionamento fino admite declarar capacidade agregada superior à fisicamente instalada, prática aparentemente análoga ao sobrecompromisso de memória. A analogia é, contudo, enganadora, e a diferença é determinante: a memória é recurso reclamável, ao passo que o armazenamento escrito não o é. Uma instância que preencha o seu disco não liberta espaço ao terminar. A consequência do esgotamento do volume subjacente é igualmente severa: as escritas falham em todas as instâncias que dele dependem, e não apenas na responsável pelo consumo, o que converte um problema local em incidente de domínio alargado. Daqui decorre um requisito operacional explícito: a monitorização deve incidir sobre o espaço real disponível, com limiar de alerta, e não sobre o espaço declarado às instâncias.

Instantâneos, Clones e Modelos

- **Instantâneo:** captura do estado da instância num momento determinado, incluindo disco e, opcionalmente, memória e estado do processador.
- Implementado por cópia na escrita: o ficheiro original torna-se apenas de leitura e as escritas subsequentes são registadas num ficheiro diferencial.
- Daqui decorre uma propriedade frequentemente ignorada: instantâneos retidos por períodos longos acumulam cadeias de ficheiros diferenciais que degradam o desempenho de leitura e complicam a recuperação.
- **Clone ligado** partilha as camadas de base com a origem; **clone completo** constitui cópia independente. Um **modelo** é uma instância convertida em imagem apenas de leitura para instanciação repetida.

Um instantâneo captura o estado da instância num momento determinado, incluindo o disco e, opcionalmente, a memória e o estado do processador. A implementação recorre a cópia na escrita: o ficheiro original torna-se apenas de leitura e as escritas subsequentes são registadas num ficheiro diferencial. Daqui decorre uma propriedade frequentemente ignorada na prática, e objeto da questão seguinte: instantâneos retidos por períodos longos acumulam cadeias de ficheiros diferenciais que degradam o desempenho de leitura e complicam a recuperação. Distinguem-se ainda o clone ligado, que partilha as camadas de base com a origem, o clone completo, cópia independente, e o modelo, instância convertida em imagem apenas de leitura destinada a instanciação repetida.

Pergunta 4

Instantâneos como cópia de segurança

Uma equipa mantém instantâneos diários das suas instâncias, retidos durante três meses, e considera-os cópia de segurança. Que objeção se impõe?

- A) Nenhuma. Um instantâneo constitui cópia de segurança completa.
- B) Os instantâneos dependem do ficheiro de base original: a sua perda ou corrupção invalida toda a cadeia, que além disso degrada o desempenho ao acumular.
- C) Os instantâneos só podem ser criados com a instância desligada.
- D) A objeção é apenas de custo de armazenamento.

A quarta questão examina uma prática corrente e incorreta, a utilização de instantâneos como cópia de segurança. A resposta decorre diretamente do mecanismo de implementação descrito no slide anterior.

Resposta 4

Opção correta: B

Um instantâneo não constitui cópia autónoma: regista diferenças relativamente a um ficheiro de base que permanece indispensável. A perda desse ficheiro, ou do volume que o aloja, invalida a totalidade da cadeia, pelo que não protege contra o cenário que uma cópia de segurança deve cobrir. Acresce a degradação progressiva do desempenho de leitura com o comprimento da cadeia. Uma cópia de segurança exige transferência do conteúdo para suporte independente, com verificação periódica do procedimento de reposição.

A fundamentação consta do slide anterior. O critério que distingue instantâneo de cópia de segurança admite formulação simples: uma cópia de segurança é autónoma relativamente ao original, um instantâneo não. Acresce a exigência, frequentemente negligenciada, de verificar periodicamente o procedimento de reposição, dado que uma cópia cuja reposição nunca foi ensaiada constitui garantia meramente presumida.

Migração em Execução (1/2)

- Consiste na transferência de uma instância em execução entre anfitriões físicos sem interrupção perceptível do serviço.
- O estado a transferir decompõe-se em três elementos: os *ficheiros* de disco, com frequência já acessíveis a ambos os anfitriões por armazenamento partilhado; a *configuração* da instância; e o *contexto* de execução, constituído por memória e estado do processador.
- Fases: reserva de recursos no destino, transferência iterativa da memória com a instância ainda ativa, suspensão breve para transferir o estado residual, e retoma no destino com reconfiguração do encaminhamento de rede.
- Constitui o mecanismo que viabiliza manutenção de hardware sem interrupção de serviço, requisito do Tier III analisado na Aula 02.

A migração em execução consiste na transferência de uma instância ativa entre anfitriões físicos sem interrupção perceptível do serviço. O estado a transferir decompõe-se em três elementos: os ficheiros de disco, com frequência já acessíveis a ambos os anfitriões por armazenamento partilhado, o que dispensa a sua transferência; a configuração da instância; e o contexto de execução, constituído pela memória e pelo estado do processador. O procedimento desenvolve-se em quatro fases: reserva de recursos no destino, transferência iterativa da memória com a instância ainda ativa, suspensão breve para transferir o estado residual, e retoma no destino com reconfiguração do encaminhamento de rede. Constitui o mecanismo que viabiliza a manutenção de hardware sem interrupção de serviço, requisito do Tier III analisado na Aula 02.

Migração em Execução (2/2)

- **Pré-cópia:** a memória é transferida iterativamente enquanto a instância executa; cada iteração transfere as páginas modificadas durante a anterior. A suspensão ocorre quando o residual é suficientemente reduzido.
- A convergência depende da relação entre a taxa de modificação de páginas e a largura de banda disponível. Cargas com escrita intensiva podem não convergir, forçando suspensão com residual elevado.
- **Pós-cópia:** a instância é retomada no destino com estado mínimo, sendo as páginas em falta obtidas por exceção. Reduz o tempo de suspensão, mas uma falha de rede durante a transferência torna a instância irrecuperável.
- A pré-cópia constitui o regime adotado por omissão, precisamente por manter a origem íntegra até à conclusão.

Duas abordagens distinguem-se pelo momento em que a instância é retomada no destino. Na pré-cópia, a memória é transferida iterativamente enquanto a instância executa, transferindo cada iteração as páginas modificadas durante a anterior, e a suspensão ocorre quando o residual é suficientemente reduzido. A convergência depende da relação entre a taxa de modificação de páginas e a largura de banda disponível, questão desenvolvida na questão seguinte. Na pós-cópia, a instância é retomada no destino com estado mínimo, sendo as páginas em falta obtidas por exceção à medida que são acedidas. Reduz-se o tempo de suspensão, mas uma falha de rede durante a transferência torna a instância irrecuperável, dado que o estado se encontra repartido por dois anfitriões. A pré-cópia constitui o regime adotado por omissão, precisamente por manter a origem íntegra até à conclusão.

Pergunta 5

Convergência da migração

Uma instância que executa uma base de dados com escrita intensiva não conclui a migração por pré-cópia, permanecendo em iterações sucessivas sem redução do residual. Qual a causa?

- A) Insuficiência de memória no anfitrião de destino.
- B) A taxa de modificação de páginas aproxima-se ou excede a largura de banda de transferência, pelo que cada iteração encontra tantas páginas modificadas quantas transferiu.
- C) O formato de disco é incompatível com migração.
- D) A pré-cópia não admite instâncias com base de dados.

A quinta questão examina a condição de convergência da migração por pré-cópia, a partir de um cenário de falha concreto. A resolução requer a identificação da relação entre as duas grandezas em causa.

Resposta 5

Opção correta: B

A pré-cópia converge apenas quando a largura de banda excede a taxa de modificação de páginas. Não se verificando essa condição, cada iteração encontra um conjunto de páginas modificadas de dimensão comparável ao que acabou de transferir, e o residual não diminui. As respostas práticas consistem em aumentar a largura de banda dedicada, impor um limite de iterações após o qual se força a suspensão, aceitando indisponibilidade mais prolongada, ou recorrer à pós-cópia.

A fundamentação consta do slide anterior. Vale registrar as três respostas operacionais possíveis, enunciadas na resolução: aumentar a largura de banda dedicada à migração, impor um limite de iterações após o qual se força a suspensão aceitando indisponibilidade mais prolongada, ou recorrer à pós-cópia. A escolha entre elas depende da tolerância do serviço a uma interrupção breve mas determinística.

Sobrecarga de Virtualização

- A camada de abstração introduz custo mensurável, hoje reduzido mas não nulo, concentrado nas transições para o monitor e na tradução de endereços.
- Cargas dominadas por cálculo aproximam-se do desempenho nativo, por executarem maioritariamente instruções inócuas sem mediação.
- Cargas dominadas por I/O ou sensíveis à latência apresentam degradação superior, decorrente da frequência das transições.
- Cenários em que a execução direta sobre o hardware permanece justificada: bases de dados de elevado débito, computação de alto desempenho e sistemas com requisitos de latência determinística.

A camada de abstração introduz custo mensurável, hoje reduzido mas não nulo, concentrado nas transições para o monitor e na tradução de endereços. A distribuição desse custo não é uniforme entre cargas. Cargas dominadas por cálculo aproximam-se do desempenho nativo, por executarem maioritariamente instruções inócuas sem mediação. Cargas dominadas por I/O ou sensíveis à latência apresentam degradação superior, decorrente da frequência das transições. Permanecem por isso cenários em que a execução direta sobre o hardware se justifica, designadamente bases de dados de elevado débito, computação de alto desempenho e sistemas com requisitos de latência determinística.

KVM

- O **KVM** é um módulo do núcleo Linux que expõe as extensões de virtualização do processador, convertendo o núcleo em hypervisor de tipo 1.
- A arquitetura reparte responsabilidades: o KVM trata da execução do convidado e da gestão de memória; o **QEMU**, em espaço de utilizador, trata da emulação de dispositivos.
- Cada instância corresponde, no anfitrião, a um processo convencional, escalonado pelo escalonador do Linux e observável pelos instrumentos habituais do sistema.
- Constitui a base tecnológica do Proxmox VE e de grande parte das infraestruturas de cloud assentes em Linux.

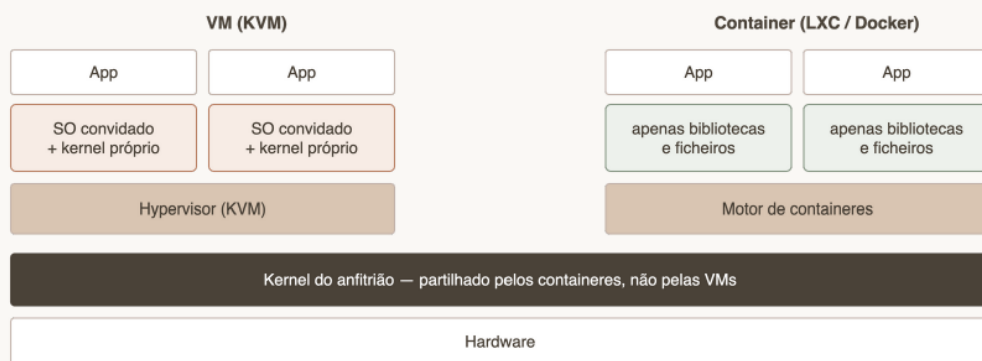
O KVM é um módulo do núcleo Linux que expõe as extensões de virtualização do processador, convertendo o núcleo em hypervisor de tipo um. A arquitetura reparte responsabilidades entre dois componentes: o KVM trata da execução do convidado e da gestão de memória, e o QEMU, em espaço de utilizador, trata da emulação de dispositivos. Uma consequência desta arquitetura merece registo: cada instância corresponde, no anfitrião, a um processo convencional, escalonado pelo escalonador do Linux e observável pelos instrumentos habituais do sistema, o que simplifica consideravelmente a monitorização e o diagnóstico. Constitui a base tecnológica do Proxmox VE e de grande parte das infraestruturas de cloud assentes em Linux.

LXC e a Virtualização ao Nível do Sistema Operativo

- O **LXC** constitui instâncias que partilham o núcleo do anfitrião, dispondo de sistema de ficheiros, espaço de processos e pilha de rede próprios.
- Não existe hardware virtual nem núcleo convidado, do que decorrem tempo de arranque na ordem dos segundos e consumo de memória limitado ao espaço de utilizador.
- O isolamento assenta em mecanismos do núcleo Linux, designadamente *namespaces* e *cgroups*, objeto da Aula 04.
- Restrições correspondentes: apenas sistemas Linux são executáveis, e o isolamento é inferior ao de uma máquina virtual por partilha da superfície de ataque do núcleo.

O LXC constitui instâncias que partilham o núcleo do anfitrião, dispondo de sistema de ficheiros, espaço de processos e pilha de rede próprios. Não existindo hardware virtual nem núcleo convidado, o tempo de arranque situa-se na ordem dos segundos e o consumo de memória limita-se ao espaço de utilizador. O isolamento assenta em mecanismos do núcleo Linux, designadamente namespaces e cgroups, que constituem o objeto da Aula 04. As restrições correspondentes decorrem da mesma propriedade: apenas sistemas Linux são executáveis, e o isolamento é inferior ao de uma máquina virtual, por partilha da superfície de ataque do núcleo, questão retomada na questão 6.

O que Cada Modelo Duplica



É por isso que o container mostra o mesmo kernel do anfitrião e a VM não.

O diagrama torna explícita a fronteira de duplicação introduzida no slide 6. Na máquina virtual, cada instância replica o núcleo e o conjunto de controladores de dispositivo. No container, essas camadas são partilhadas com o anfitrião e apenas o espaço de utilizador é replicado. É esta diferença que explica simultaneamente a disparidade de tempo de arranque, de consumo de memória e de densidade de instâncias por anfitrião.

Comparação dos Dois Modelos

	Máquina virtual	Container
Núcleo	Próprio por instância	Partilhado com o anfitrião
Arranque	Dezenas de segundos	Segundos ou menos
Sobrecarga	Núcleo e dispositivos	Praticamente nula
Sistema convidado	Qualquer	Apenas Linux
Isolamento	Fronteira de hardware	Fronteira de núcleo
Densidade típica	Dezenas por anfitrião	Centenas por anfitrião

- Os modelos não são alternativos mas complementares: é prática corrente executar containers dentro de máquinas virtuais, combinando o isolamento das primeiras com a densidade dos segundos.

A tabela sistematiza a comparação em seis dimensões. A leitura conjunta das linhas revela que as diferenças não são independentes entre si: decorrem todas da mesma decisão quanto à fronteira de duplicação. A última observação do slide merece ênfase, por corrigir um equívoco frequente: os modelos não são alternativos mas complementares, sendo prática corrente executar containers dentro de máquinas virtuais, combinação que associa o isolamento das primeiras à densidade dos segundos. O laboratório desta aula torna essa combinação observável em equipamento próprio.

Pergunta 6

Isolamento entre clientes distintos

Um fornecedor pretende alojar cargas de clientes distintos, sem relação de confiança entre si, no mesmo equipamento físico. Que modelo se justifica como fronteira primária de isolamento?

- A) Containers, por apresentarem menor sobrecarga e maior densidade.
- B) Máquinas virtuais, por a fronteira de isolamento ser assegurada pelo hardware e não pelo núcleo partilhado, cuja superfície de ataque é comum a todos os containers.
- C) Indiferente: ambos oferecem isolamento equivalente.
- D) Containers, desde que executados com utilizador não privilegiado.

A sexta questão examina a seleção do modelo adequado a um cenário multi-cliente sem relação de confiança. A resposta decorre da natureza da fronteira de isolamento de cada modelo, e não das suas características de desempenho.

Resposta 6

Opção correta: B

Containers no mesmo anfitrião partilham o núcleo, pelo que uma vulnerabilidade que permita escapar ao isolamento compromete todas as instâncias. A máquina virtual estabelece a fronteira ao nível do hardware, com superfície de ataque substancialmente reduzida. A opção D descreve uma medida de mitigação pertinente, mas que não altera a natureza da fronteira. A prática corrente na indústria consiste em combinar ambos: uma máquina virtual por cliente, com containers no seu interior.

A fundamentação consta do slide anterior. Importa reter que a opção D, a execução de containers com utilizador não privilegiado, constitui medida de mitigação pertinente e recomendável, mas que não altera a natureza da fronteira: o núcleo permanece partilhado. A prática corrente na indústria, referida na resolução, consiste em combinar ambos os modelos, atribuindo uma máquina virtual por cliente e executando containers no seu interior.

Proxmox VE

- Plataforma de virtualização que integra numa interface única os dois modelos: KVM para máquinas virtuais e LXC para containers de sistema.
- Gestão por interface web ou por linha de comandos, através de `qm` para instâncias KVM e `pct` para containers LXC.
- Suporta instantâneos, clones, modelos e migração entre nós, concretizando os mecanismos analisados nesta aula.
- Constitui a plataforma que sustenta o trabalho de projeto da segunda metade do semestre.

O Proxmox VE integra numa interface única os dois modelos analisados, o KVM para máquinas virtuais e o LXC para containers de sistema, o que o torna adequado a fins pedagógicos por permitir a comparação direta entre ambos. A gestão faz-se por interface web ou por linha de comandos, através de `qm` para instâncias KVM e `pct` para containers LXC. A plataforma suporta instantâneos, clones, modelos e migração entre nós, concretizando os mecanismos analisados ao longo desta aula, e constitui o ambiente que sustenta o trabalho de projeto da segunda metade do semestre.

Laboratório: Primeiro Contacto com Containers

Trabalho em equipamento próprio

O laboratório associado a esta aula destina-se a tornar observável, no computador de cada aluno, a distinção entre virtualização completa e virtualização ao nível do sistema operativo. Utiliza Docker por simplicidade de instalação, sendo os mecanismos de núcleo subjacentes matéria da Aula 04.

- Realiza-se durante a aula, sem necessidade de acesso ao servidor da unidade curricular.
- Não constitui trabalho a entregar. As caixas *Pare e observe* assinalam os momentos de interpretação e discussão.

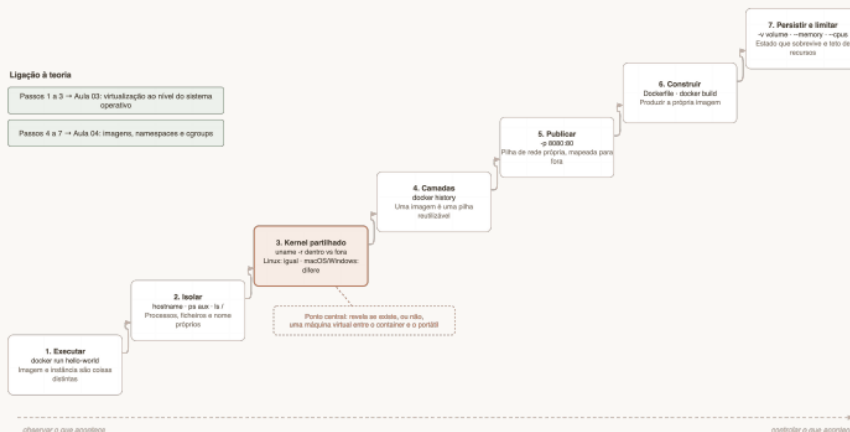
O laboratório associado a esta aula decorre em equipamento próprio de cada aluno e não requer acesso ao servidor da unidade curricular. O seu objetivo não é o domínio da ferramenta, mas a observação direta do comportamento que a exposição descreveu. Recorre a Docker por ser a implementação de containers de instalação mais simples em qualquer sistema operativo, ficando os mecanismos de núcleo subjacentes, namespaces e cgroups, para a Aula 04. Não constitui trabalho a entregar: destina-se a ser percorrido durante a aula, ao ritmo de cada aluno. As caixas *Pare e observe* distribuídas pelo guião assinalam os momentos em que interessa deter a execução e interpretar o resultado, isoladamente ou em discussão, antes de prosseguir.

Laboratório: Percurso

Percurso do laboratório: do primeiro comando ao controlo de recursos

Ligação à teoria

- Passos 1 a 3 → Aula 03: virtualização ao nível do sistema operativo
- Passos 4 a 7 → Aula 04: imagens, namespaces e cgroups



O diagrama apresenta a sequência completa do guião numa única imagem, com os passos dispostos em progressão ascendente. A leitura da esquerda para a direita acompanha o aumento de complexidade e, simultaneamente, uma mudança de natureza do trabalho: os primeiros passos limitam-se a observar o que o sistema faz, ao passo que os últimos passam a controlá-lo, construindo imagens próprias e impondo limites de recursos. O passo 3 encontra-se assinalado por constituir a observação central, aquela que revela se existe, ou não, uma máquina virtual interposta entre o container e o computador. As duas caixas do canto superior esquerdo estabelecem a correspondência com a teoria: os três primeiros passos ilustram matéria desta aula, os restantes antecipam a Aula 04.

Laboratório: os Sete Passos

- Distinção entre imagem e instância em execução, observável na primeira execução.
- Isolamento de processos, sistema de ficheiros e identificação de máquina.
- **Comparação do núcleo dentro e fora do container**, observação central do trabalho.
- Estrutura em camadas de uma imagem e reutilização entre imagens derivadas.
- Publicação de um serviço em rede e construção de imagem própria.
- Persistência de dados e imposição de limites de recursos.

A enumeração detalha os sete passos representados no diagrama anterior. Começa pela distinção entre imagem e instância em execução, análoga à distinção entre ficheiro de disco virtual e máquina virtual ativa. Prossegue com a observação do isolamento de processos, sistema de ficheiros e identificação de máquina. O terceiro ponto constitui a observação central do trabalho, a comparação da versão do núcleo dentro e fora do container, cujo resultado difere consoante o sistema operativo do computador e é objeto da questão 7. Seguem-se a estrutura em camadas de uma imagem, a publicação de um serviço em rede, a construção de imagem própria, a persistência de dados e a imposição de limites de recursos, estes últimos antecipando os cgroups da Aula 04.

Pergunta 7

Antecipação de um resultado do laboratório

Um aluno executa `uname -r` dentro de um container no seu computador com macOS e obtém a identificação de um núcleo Linux. Qual a interpretação correta?

- A) O container possui núcleo Linux próprio, independente do anfitrião.
- B) O Docker Desktop executa uma máquina virtual Linux sobre um hypervisor do sistema, e o container partilha o núcleo dessa máquina virtual.
- C) O resultado indica erro de instalação.
- D) O macOS possui um núcleo Linux interno utilizado pelos containers.

A sétima questão antecipa um resultado que o laboratório produzirá em computadores com macOS ou Windows, e cuja interpretação constitui o ponto pedagógico central do trabalho.

Resposta 7

Opção correta: B

Os containers constituem mecanismo do núcleo Linux e não são executáveis sobre o núcleo do macOS. O Docker Desktop resolve a questão instanciando uma máquina virtual Linux sobre o hypervisor do sistema, e é o núcleo dessa máquina virtual que o container partilha. Verifica-se assim, no mesmo equipamento, a coexistência dos dois modelos estudados: virtualização completa a sustentar a máquina virtual, e virtualização ao nível do sistema operativo no seu interior. Em Linux, a mesma verificação devolveria o núcleo do próprio anfitrião.

A fundamentação consta do slide anterior. O resultado merece ser retido pelo que revela: em Windows e macOS coexistem, no mesmo equipamento, os dois modelos de virtualização estudados nesta aula, a virtualização completa que sustenta a máquina virtual Linux e a virtualização ao nível do sistema operativo que produz o container no seu interior. A afirmação corrente de que containers dispensam máquinas virtuais é, nestes sistemas, incorreta. Em Linux, a mesma verificação devolveria o núcleo do próprio anfitrião, confirmando a partilha direta.

Síntese da Aula (1/2)

- A virtualização aplica-se a três níveis distintos, diferenciados pela fronteira de duplicação entre instâncias: hardware, sistema operativo e aplicação.
- O teorema de Popek e Goldberg estabelece a condição de virtualizabilidade de uma arquitetura; o x86 não a satisfazia, do que decorreram três respostas distintas.
- Tradução binária, paravirtualização e assistência de hardware resolvem a mesma limitação, diferindo na necessidade de modificar o convidado e no suporte exigido ao processador.
- A virtualização de memória evoluiu das tabelas sombra, de manutenção dispendiosa, para a paginação em dois níveis assistida por hardware.

A primeira metade da síntese retoma os resultados relativos à fundamentação e à virtualização de processador e memória. A virtualização aplica-se a três níveis diferenciados pela fronteira de duplicação. O teorema de Popek e Goldberg estabelece a condição de virtualizabilidade de uma arquitetura, que o x86 não satisfazia, do que decorreram três respostas historicamente distintas. Tradução binária, paravirtualização e assistência de hardware resolvem a mesma limitação, diferindo na necessidade de modificar o convidado e no suporte exigido ao processador. A virtualização de memória evoluiu das tabelas sombra, de manutenção dispendiosa, para a paginação em dois níveis assistida por hardware.

Síntese da Aula (2/2)

- O I/O admite emulação, interface paravirtualizada virtio, atribuição exclusiva por passthrough ou partilha por SR-IOV, com compromissos entre desempenho e flexibilidade.
- O sobrecompromisso é admissível em memória, recurso reclamável, e exige vigilância acrescida em armazenamento, onde o espaço escrito não é recuperável.
- A migração por pré-cópia converge apenas quando a largura de banda excede a taxa de modificação de páginas.
- Máquinas virtuais e containers diferem na fronteira de isolamento, do hardware ou do núcleo, o que determina a sua aplicação em cenários multi-cliente.

A segunda metade retoma os resultados relativos ao I/O, ao armazenamento e à operação. O I/O admite quatro tratamentos, com compromissos entre desempenho e flexibilidade. O sobrecompromisso é admissível em memória, recurso reclamável, e exige vigilância acrescida em armazenamento, onde o espaço escrito não é recuperável. A migração por pré-cópia converge apenas quando a largura de banda excede a taxa de modificação de páginas. Por último, máquinas virtuais e containers diferem na natureza da fronteira de isolamento, do hardware ou do núcleo, o que determina a sua aplicação em cenários multi-cliente e justifica a sua combinação frequente.

Obrigada.

Questões e comentários

Conclui-se a exposição da Aula 03. O laboratório associado deve ser percorrido durante a aula, antes da Aula 04, que estabelece os mecanismos de núcleo responsáveis pelo comportamento nele observado: os namespaces, que produzem o isolamento constatado, e os cgroups, que impõem os limites de recursos aplicados. As observações feitas servirão de referência concreta para essa exposição.